

Система квестов

Отчёт о разработке учебного проекта

QuestDesk

Планы. Действия. Результаты.

Python + Tkinter/ttk + PostgreSQL

8 таблиц / 6 экранов / 20 автоматических тестов

Тема: разработка настольной системы личных квестов, развития навыков и получения наград.

Выполнили: _____

Группа: _____

Преподаватель: _____

Поля участников предназначены для заполнения реальными данными перед сдачей. Распределение ролей приведено в разделе 7.

2026

1. Концепция и требования

Пользовательский результат и границы проекта

Идея и цель

QuestDesk превращает учебные и личные планы в квесты с проверяемыми шагами. Выполнение изменяет состояние профиля: опыт, уровень, монеты и навыки. Монеты обмениваются на личные награды.

Функциональные требования

1. Создание, просмотр, редактирование и удаление квестов.
2. Поиск по названию и фильтрация по состоянию.
3. Начало квеста, выполнение шагов, завершение и отказ.
4. Начисление опыта и монет; автоматическая обработка просрочки.
5. Управление навыками, наградами и связями квест-навык.
6. Покупка награды с проверкой баланса и журналом операций.
7. Сохранение состояния после закрытия и повторного запуска.

Нефункциональные требования

Настольный русскоязычный интерфейс для Windows x64, PostgreSQL, параметризованные запросы, ограничения целостности, атомарные операции, проверка ввода и воспроизводимые тесты.

Границы системы

Один локальный профиль без регистрации. Для первого запуска поставляется seed: четыре осмысленных квеста, десять шагов, три навыка и три награды. Локальный сервер запускается автоматически; интернет для готовой сборки не нужен.

2. Архитектура

Разделение интерфейса, правил и хранения

Слои приложения

app.py - интерфейс Tkinter/ttk: навигация, формы, карточки и сообщения. Вызывает сервис и отображает результат.

domain.py - независимые правила: величины наград, вычисление уровня, валидация текста и допустимость состояния.

service.py - бизнес-операции: переходы состояний, транзакции, блокировки, начисление, покупки и события.

db.py - соединения Psycopg, выполнение запросов и инициализация схемы. **bootstrap.py** - подготовка и запуск локальной PostgreSQL.

Поток выполнения

Действие в интерфейсе → проверка ввода → сервис → транзакция PostgreSQL → обновление состояния → отображение результата. Ошибка приводит к откату операции и понятному сообщению пользователю.

Почему выбрана такая структура

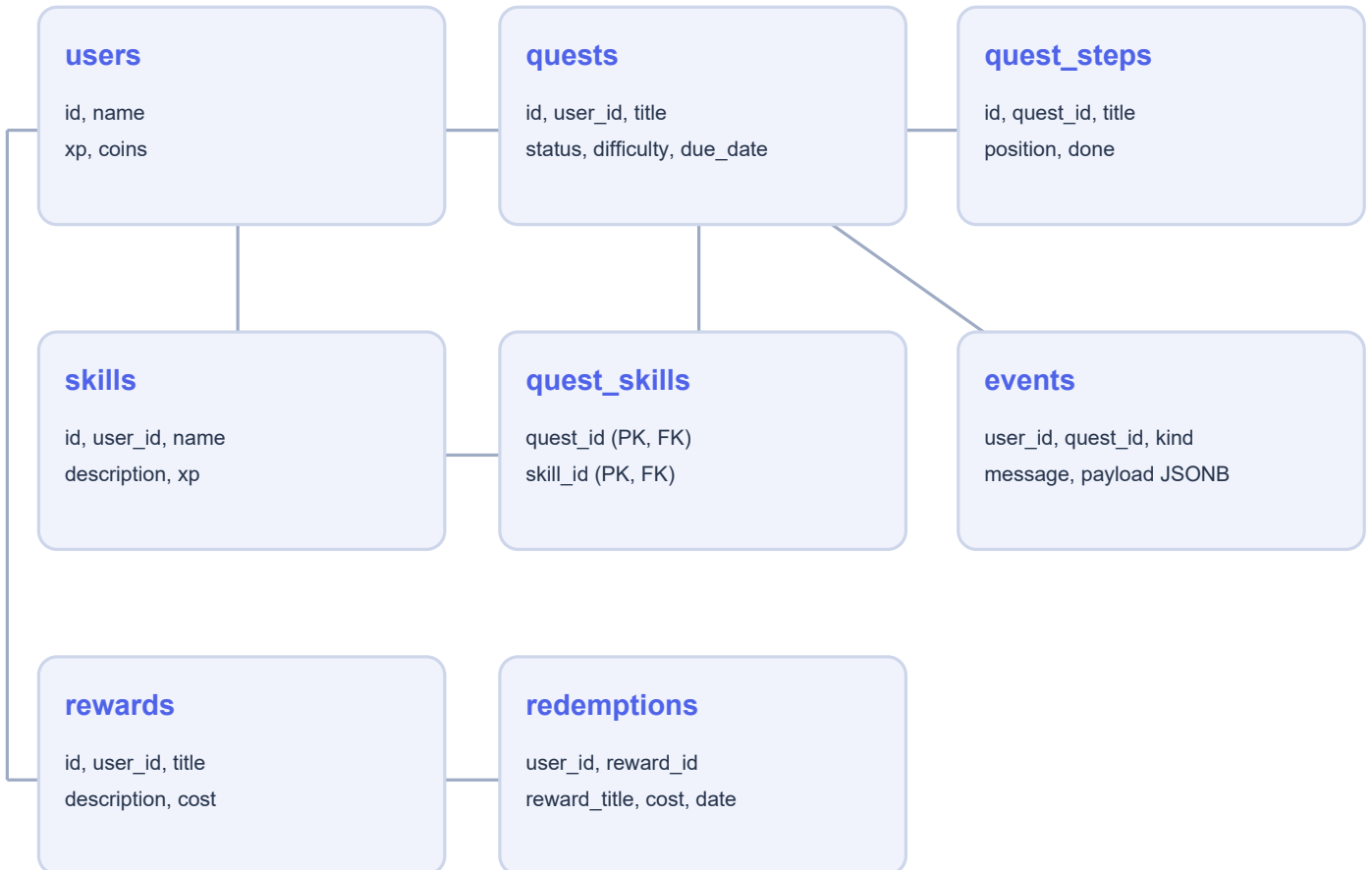
Бизнес-правила можно проверять без интерфейса. Изменение дизайна не меняет начисление опыта. Конкурентные действия контролируются на уровне транзакций, а не временным отключением кнопки.

Работа с данными

Каждое соединение короткоживущее. Успешный выход из контекста соединения фиксирует изменения; исключение откатывает их. SQL-параметры передаются отдельно от текста запроса. Тайм-аут запроса - 10 секунд, ожидания блокировки - 5 секунд.

3. Схема PostgreSQL

Восемь связанных таблиц; полная DDL находится в `schema.sql`



1:N: профиль - квесты; квест - шаги. **N:M:** квесты - навыки через `quest_skills`. На диаграмме показаны основные связи; все внешние ключи перечислены в DDL и `docs/architecture.md`.

CHECK: неотрицательные опыт/баланс, допустимая цена, непустые названия, согласованность статуса и даты завершения. **UNIQUE:** имя навыка и награды в профиле, позиция шага. **ENUM:** статус и сложность.

Индексы: по профилю/статусу/сроку, по времени событий и покупок, обратным связям; GIN по JSONB. JSONB содержит изменяемый состав деталей событий; основные сущности остаются реляционными.

4. Бизнес-логика

Состояния, награды, последствия и защита от повторов

Жизненный цикл квеста

Запланирован → В работе → Завершён / Не выполнен. Редактируется только запланированный квест. Активный квест нельзя удалить без завершения или отказа. Все шаги должны быть выполнены до начисления награды.

Расчёт прогресса

Лёгкий квест: 40 XP и 15 монет. Обычный: 80 XP и 30 монет. Сложный: 140 XP и 50 монет.
Уровень = 1 + целая часть (XP / 200). Каждый выбранный навык получает полный опыт завершённого квеста.

Сроки и штрафы

Дата срока включительна. Активный квест с датой раньше сегодняшней становится невыполненным. Штраф равен минимуму из 10 монет и текущего баланса. Повторная обработка не штрафует снова. Проверка выполняется при открытии, навигации, обновлении и раз в минуту.

Транзакция завершения

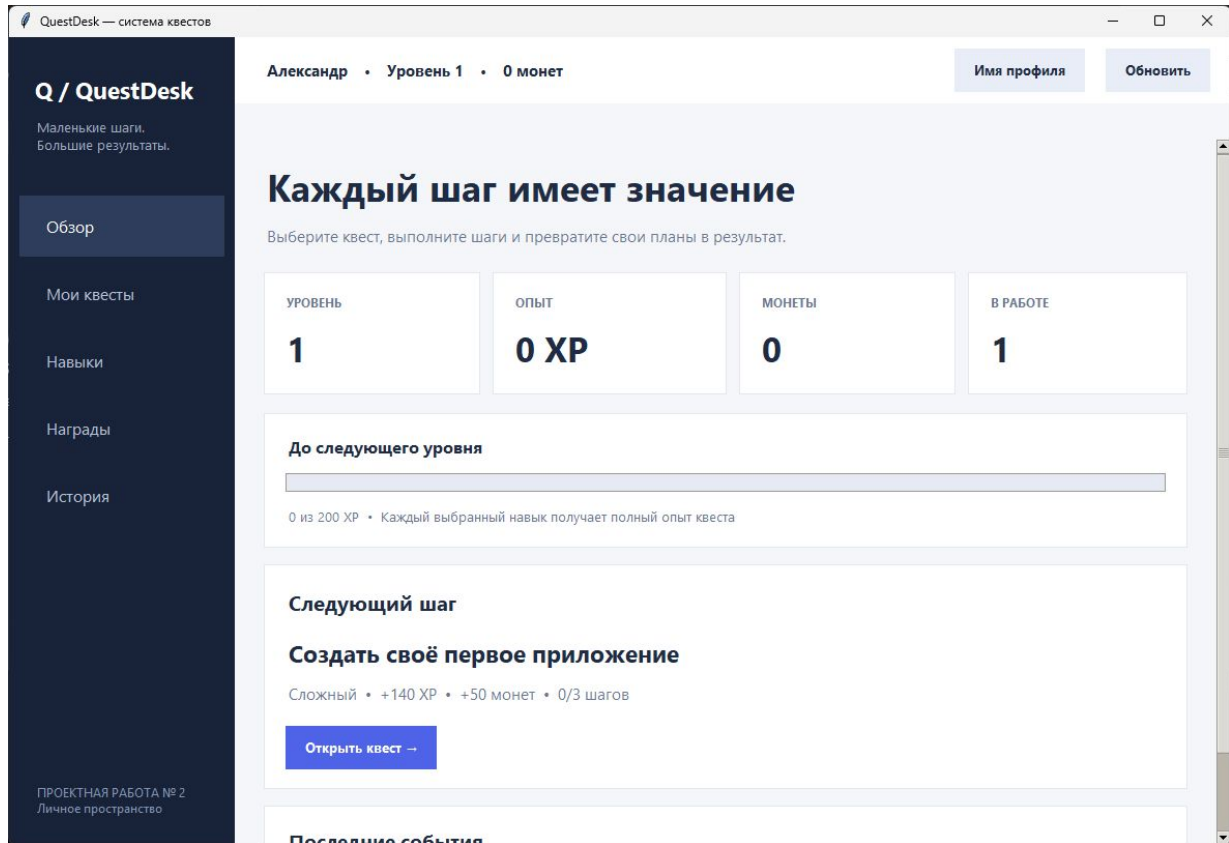
Блокировка профиля SELECT FOR UPDATE → проверка актуального статуса и шагов → изменение квеста → начисление профилю → начисление навыкам → запись события → COMMIT. Ошибка откатывает все изменения.

Покупки и история

Покупка блокирует тот же профиль и проверяет актуальный баланс. Списание и запись факта покупки атомарны. Название и цена сохраняются как снимок: редактирование или удаление каталога не переписывает прошлое. Повторная покупка разрешена при достаточном балансе.

5. Интерфейс и сценарии

Минималистичная навигация и понятные действия



Обзор: состояние профиля, прогресс до уровня, следующий квест. **Мои квесты:** список, поиск и фильтр. **Карточка:** описание и взаимодействие с шагами. **Навыки, Награды, История:** развитие, покупки и обратная связь.

Основной сценарий: открыть первый квест → отметить три шага → завершить → получить 140 XP и 50 монет → купить награду за 30 → увидеть 20 монет и историю.

Альтернативные сценарии: преждевременное завершение отклоняется; недостаток монет не создаёт покупку; некорректная дата не сохраняется; удаление связанного навыка блокируется.

6. Проверка качества

Автоматические тесты и проверка настольного интерфейса

Результат выполненного прогона

20 тестов пройдены. Команда: `python -m pytest -q`. Тесты бизнес-операций выполнялись на настоящей PostgreSQL 17.6, а не на подмене хранилища.

Покрытые сценарии

- Границы уровней и проверка непустых названий.
- Инициализация схемы без повторного seed.
- Создание, изменение, поиск и удаление квеста.
- Начисление профилю и навыкам, запись JSONB-события.
- Отказ в завершении при незаконченных шагах.
- Два одновременных завершения: награда только одна.
- Две одновременные покупки: баланс не уходит в минус.
- Просрочка и отказ: штраф однократный и ограниченный.
- Защита активного квеста от изменения и удаления.
- CRUD справочников и снимки истории покупок.
- Валидация сроков, SQL-инъекция, CHECK и UNIQUE.
- Удаление квеста сохраняет опыт и события.

Изоляция тестов

Каждый интеграционный тест создаёт отдельную базу `questdesk_test_<uuid>` и удаляет её после проверки. Рабочая база приложения не очищается. Для запуска необходимы права CREATEDB.

Интерфейс

Программно проверено открытие обзора, списка, карточки, навыков, наград, истории и форм. Выполнена визуальная проверка настольного окна. Проверка на других версиях Windows отдельно не выполнялась.

7. Запуск и защита

Комплект сдачи и распределение ответственности

Комплект

Исходный код, SQL-схема и seed, автоматические тесты, готовая Windows-сборка, README, схема архитектуры, этот отчёт и сценарий защиты docs/defense.md.

Как запустить

Открыть dist/QuestDesk/QuestDesk.exe. Переносить всю папку сборки вместе с _internal и vendor. Python и PostgreSQL включены. Личные данные размещаются в %LOCALAPPDATA%/QuestDesk и сохраняются между запусками. Сервер слушает 127.0.0.1:55439.

Распределение ролей для группы

Backend / бизнес-логика: domain.py и service.py.

Работа с БД: schema.sql, seed.sql, ограничения и индексы.

UI/UX: app.py, навигация и формы.

Интеграция и тестирование (опционально): bootstrap.py, tests, сборка. ФИО и фактический вклад заполняются участниками.

План демонстрации

Показать состояние → выполнить квест → проверить опыт навыков → купить награду → продемонстрировать отказ при нехватке монет → открыть историю → перезапустить программу → объяснить связи и транзакцию.

Источники и дальнейшее развитие

Исходное задание: «Проектная работа № 2. Разработка настольного приложения с бизнес-логикой и системой данных», 6 страниц.

PostgreSQL: <https://www.postgresql.org/docs/17/explicit-locking.html>

Psycopg: <https://www.psycopg.org/psycopg3/docs/basic/transactions.html>

Дальнейшее развитие: авторизация нескольких профилей, миграции схемы, резервные копии и асинхронные запросы для удалённой БД.